

HACKSIMUL8 2026 — Submission Handoff

6-DOF UAV Quadcopter: modelling, PID altitude control, ML-based self-tuning PES University ·
Department of Mechanical Engineering · 2 September 2026

Quick start — reproduce everything in 3 commands

```
cd 'C:\Users\LENOVO\Downloads\HACKSIMUL8\solution'
smoke_test      % 10-point sanity check, ~30 s
verify_all      % 25 independent verification checks, ~2 min
```

To open the Simulink model:

```
open_system('quad_altitude')    % press Run, then double-click the Scope
```

Status

Task	Requirement	Status
1	Dynamic model in state-space, MATLAB + Simulink	✓ Complete, verified
2	Altitude PID + tuning methodology	✓ Complete, benchmarked vs the cited paper
3	Generate data + train ML model for self-tuning	✓ Complete
4	Implement model, test cases with comparisons	⚠ In progress

Dependency order is 1 → 2 → 3 → 4 . Tasks 1–3 stand alone and are fully demonstrable.

Headline results

Result	Value
Model verification ($\max\ A - A_{\text{numerical}}\ $)	1.6e-12
Controllability	rank 12 of 12
Final PID gains	Kp = 30.1847, Ki = 0.0000, Kd = 10.3994
Settling time	0.95 s (reference paper's best: <5 s)
Overshoot	0.00% (reference paper's best: <5%)
Altitude sag at 17.2° pitch	0.000003 m
Optimal Ki vs payload	0 → 5.50 → 11.01 → 16.05 for mass ×1.0 → ×2.2

Documentation — read in this order

Document	Contents
TASK1_final.md / .pdf	Model derivation, state space, verification, file guide, presentation script, judge Q&A
TASK2_final.md / .pdf	Feedback linearisation, four tuning methods, reference-paper benchmark, file guide, Q&A
TASK3_final.md / .pdf	Dataset design, model selection, the Ki-vs-payload finding, defects found and fixed, Q&A
04_SOLUTION_PLAN.md	Original strategy document (historical)
05 – 07	Earlier working drafts, superseded by the _final versions
01 – 03	MATLAB/Simulink course notes (preparation material)

MATLAB files — what each one does

Core model (Task 1)

File	Purpose
quad_params.m	All 8 Table 1 parameters plus derived quantities. Single source of truth — nothing is hard-coded elsewhere.
quad_dynamics.m	Full nonlinear 6-DOF model: 12 states in, x out.
rotor2U.m	Rotor speeds → the four virtual inputs U1–U4, via $\tau = r \times F$.
task1_statespace.m	Builds A,B,C,D; verifies against finite differences; controllability; open-loop divergence.

Control design (Task 2)

File	Purpose
<code>task2_altitude_pid.m</code>	Four tuning methods compared on the nonlinear model. Produces the final gains.
<code>task2b_zn_tyreus_luyben.m</code>	Benchmark against Mien & Tu (2024) — reproduces their published gains, then compares.
<code>sim_altitude.m</code>	The nonlinear closed-loop simulator used everywhere. Saturation, anti-windup, derivative-on-measurement, 100 Hz discrete control, RK4 plant.
<code>perf_metrics.m</code>	Tr, Ts, overshoot, SSE, ITAE, IAE, ISE, control effort.
<code>alt_cost.m</code>	The objective minimised by Method D and by every Task 3 label.
<code>build_simulink_model.m</code>	Constructs <code>quad_altitude.slx</code> programmatically, reading gains from <code>task2_pid.mat</code> .
<code>quad_altitude.slx</code>	The Simulink deliverable.

Machine learning (Tasks 3–4)

File	Purpose
<code>task3_generate_data_train_ml.m</code>	Samples 150 conditions, extracts features, optimises labels, trains. Uses <code>parfor</code> .
<code>task3b_retrain_compare.m</code>	Trains four model classes, selects the best on held-out data.
<code>task3c_relabel_warmstart.m</code>	Re-labels after a cost change, warm-started.
<code>task4_test_ml_selftuning.m</code>	Two-phase self-tuner, 5 test cases, three-way comparison.

Validation and output

File	Purpose
<code>smoke_test.m</code>	10-point regression check. Run after any edit.
<code>verify_all.m</code>	25 independent checks — re-derives results rather than trusting saved ones.
<code>export_figures.m</code>	Regenerates every figure as a white-background 150 dpi PNG.
<code>run_all.m</code>	Runs Tasks 1–4 in order.

Data files produced

File	Contents
task1_model.mat	A, B, C, D, sys_full, G_alt
task2_pid.mat	final gains + all four methods' metrics
task2b_benchmark.mat	reference-paper comparison
task3_dataset.mat	150×8 features, 150×3 labels, true conditions
task3_model.mat	trained model + predict_gains handle
task4_results.mat	per-test-case metrics

Figures (solution/figures/)

File	Shows
01_task1_openloop_divergence.png	2% thrust error diverging as t^2
02_task1_four_channels_pzmap.png	four double integrators, poles at origin
03_task2_tuning_comparison.png	four tuning methods overlaid
04_feedback_linearisation_tilt.png	tilt sweep, error in millimetres
05_fixed_gains_degrade.png	payload → 36 cm error (bridge to Task 3)
06_task3_gains_vs_mass.png	optimal gains against payload
07_reference_paper_benchmark.png	our design vs Mien & Tu's three methods

Reference paper

Mien, T. & Tu, T. (2024), "Design and Quality Evaluation of the Position and Attitude Control System for 6-DOF UAV Quadcopter Using Heuristic PID Tuning Methods", IJRCS 4(4), 1712–1730. doi:10.31763/ijrsc.v4i4.1594

Local copy: 1594-5282-3-PB.pdf

What we verified against it:

- Table 1 parameters — identical in all 8 values
- Our implementation of their ZN (Eq. 24) and Tyreus–Luyben (Eq. 25) formulas reproduces their published Table 2/3 altitude gains **to four decimal places**, from their published $k_{th} = 124.99$, $\tau_{th} = 3.52\text{ s}$
- Their altitude control law (Eq. 27) is a **plain PID** — no gravity feed-forward, no tilt compensation. Ours has both.
- Their Eq. (6) includes air-resistance terms A_x, A_y, A_z , but **neither their Table 1 nor our problem statement gives values**. We set them to zero, which is why our altitude channel is a pure double integrator and theirs is not.

Three things that differentiate this submission

- 1. Feedback linearisation on the altitude channel.** $U1 = m(g+u_z)/(\cos \phi \cos \theta)$ makes $\ddot{z} = u_z$ *exactly*, at any tilt — not a small-angle approximation. Verified: 0.000003 m sag at 17.2° pitch, where a hover-linearised controller loses centimetres.
- 2. The controller is never told the payload.** In hover, thrust equals weight, so mean thrust ÷ nominal weight **is** the mass ratio ($r = +0.84$ against optimal K_i). Most implementations feed the mass in as an input, which is gain scheduling, not self-tuning.
- 3. We report what the data can and cannot identify.** K_i predicts at $R^2 \approx 0.70\text{--}0.87$; K_p and K_d do not, because the cost surface has a flat valley in those directions and their labels are intrinsically noisy. Adapting only the identifiable parameter is the correct response — and we say so rather than hiding it.